What is an Event Loop in Python?

An **event loop** is the core of asynchronous programming with `asyncio`. It continuously runs, waiting for and dispatching events or tasks, enabling your program to manage multiple asynchronous operations without blocking.

How it works:

- It schedules and runs **coroutines**.
- It waits for **I/O events**, timers, or other triggers.
- It switches between tasks when they await something, so other tasks can progress.

Simple example using the event loop:

python Copiar Editar

```python
import asyncio

async def greet():
    print("Hello")
    await asyncio.sleep(1)  # Simulate an asynchronous wait
    print("World")

# Get the default event loop
loop = asyncio.get_event_loop()

# Run the coroutine until complete
loop.run_until_complete(greet())

# Close the loop when done (optional in recent Python versions)
loop.close()
```

Explanation:

- `asyncio.get_event_loop()` gets the event loop instance.
- `run_until_complete()` runs the coroutine until it finishes.
- `await asyncio.sleep(1)` pauses `greet()` without blocking the whole program, letting the event loop handle other tasks if any.

Since Python 3.7+, you can also use the simpler:

python Copiar Editar

```
asyncio.run(greet())
```

which creates and closes the event loop automatically.